Lightweight LLM Agent Memory with Small Language Models

ACL 2026 Main Conference，Volume 1: Long Papers

Group Meeting · Paper presentation
Presenter: Yu Leming
2026.10

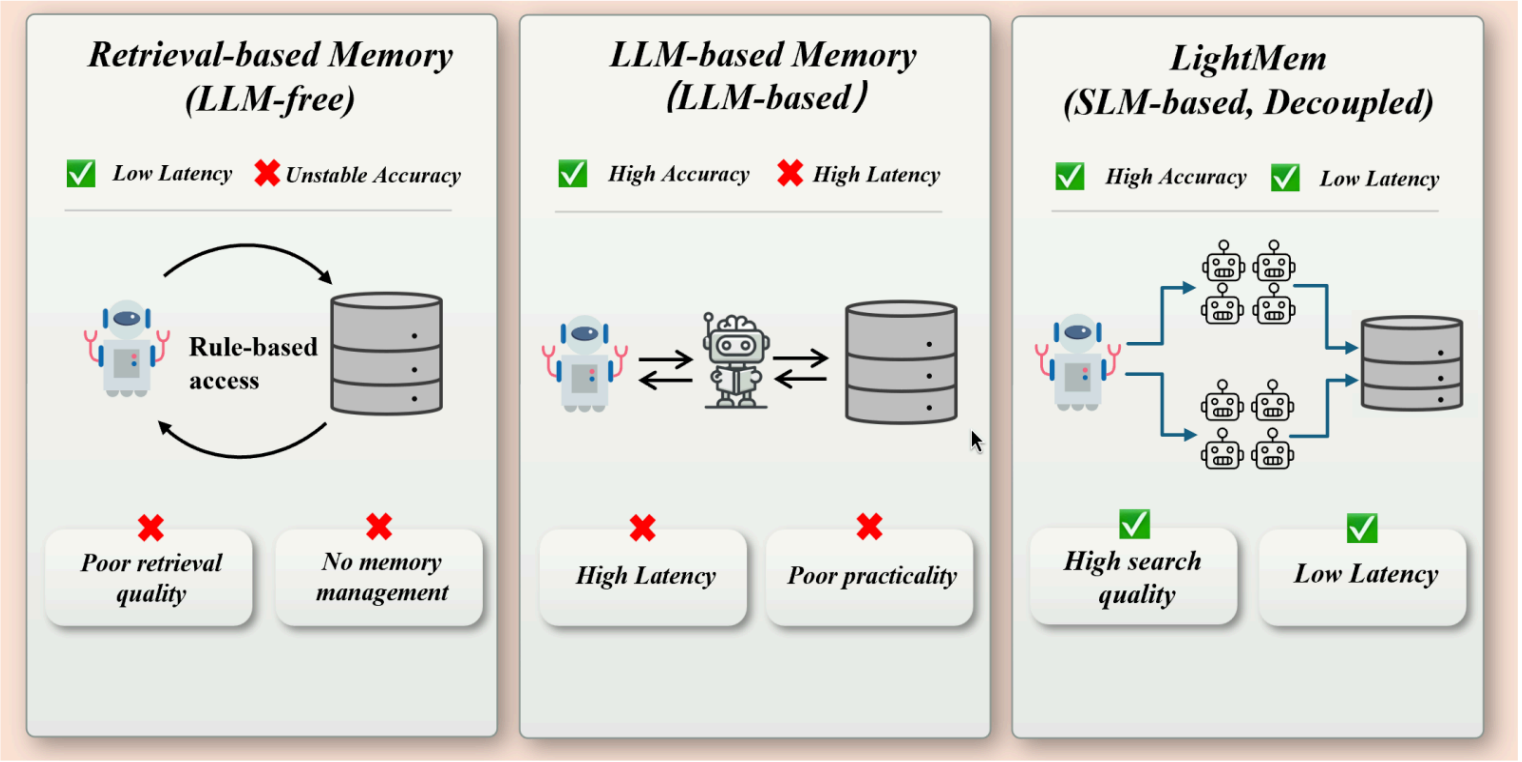
Why do agents need a better memory system?

context window

Full-history replay: simple but expensive in tokens and latency.

Vector retrieval: efficient, but noisy memories.

LLM-driven: good but repeated calls add cost.



Can a memory system improve retrieval quality while keeping online operations lightweight?

SLM & LLM driven

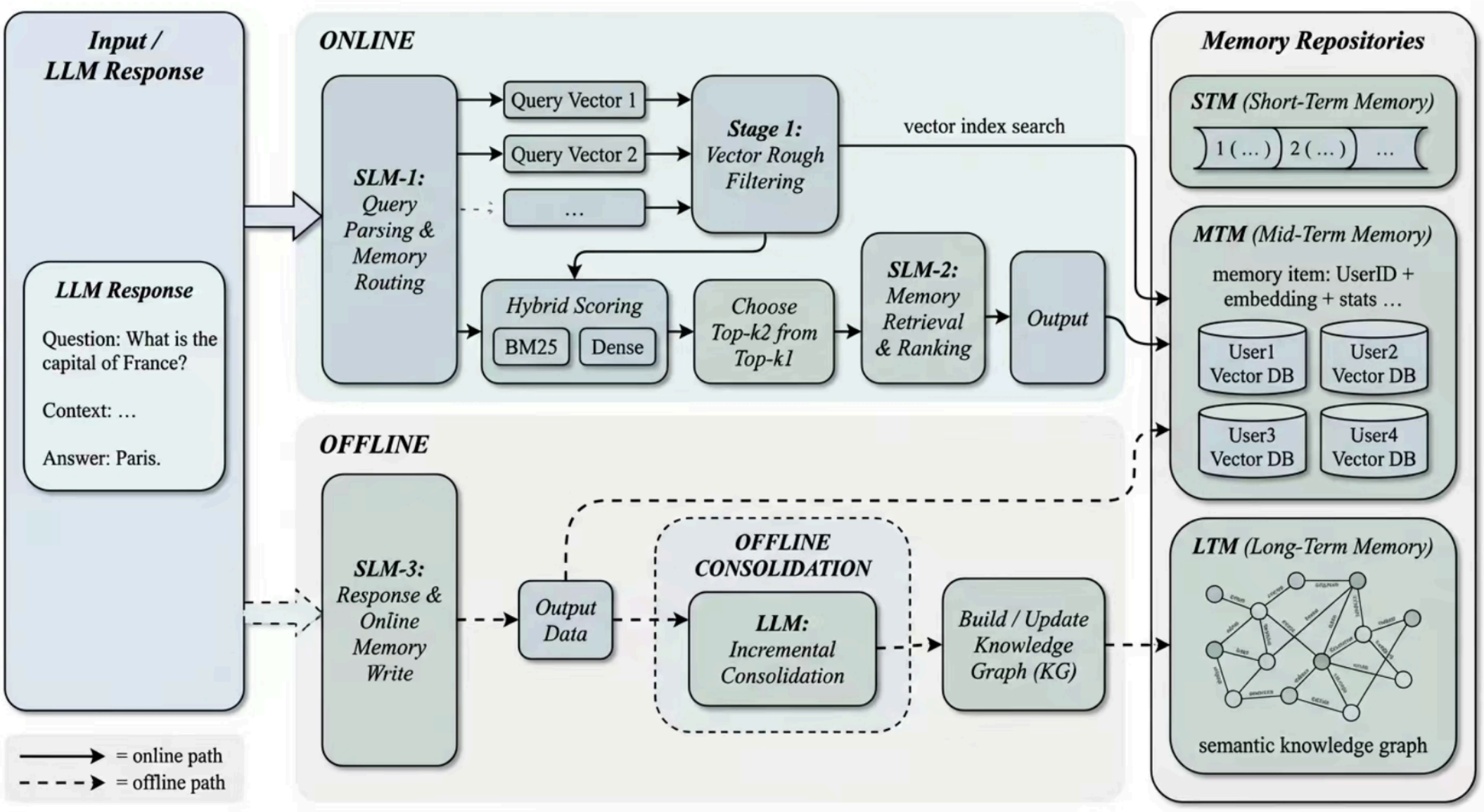


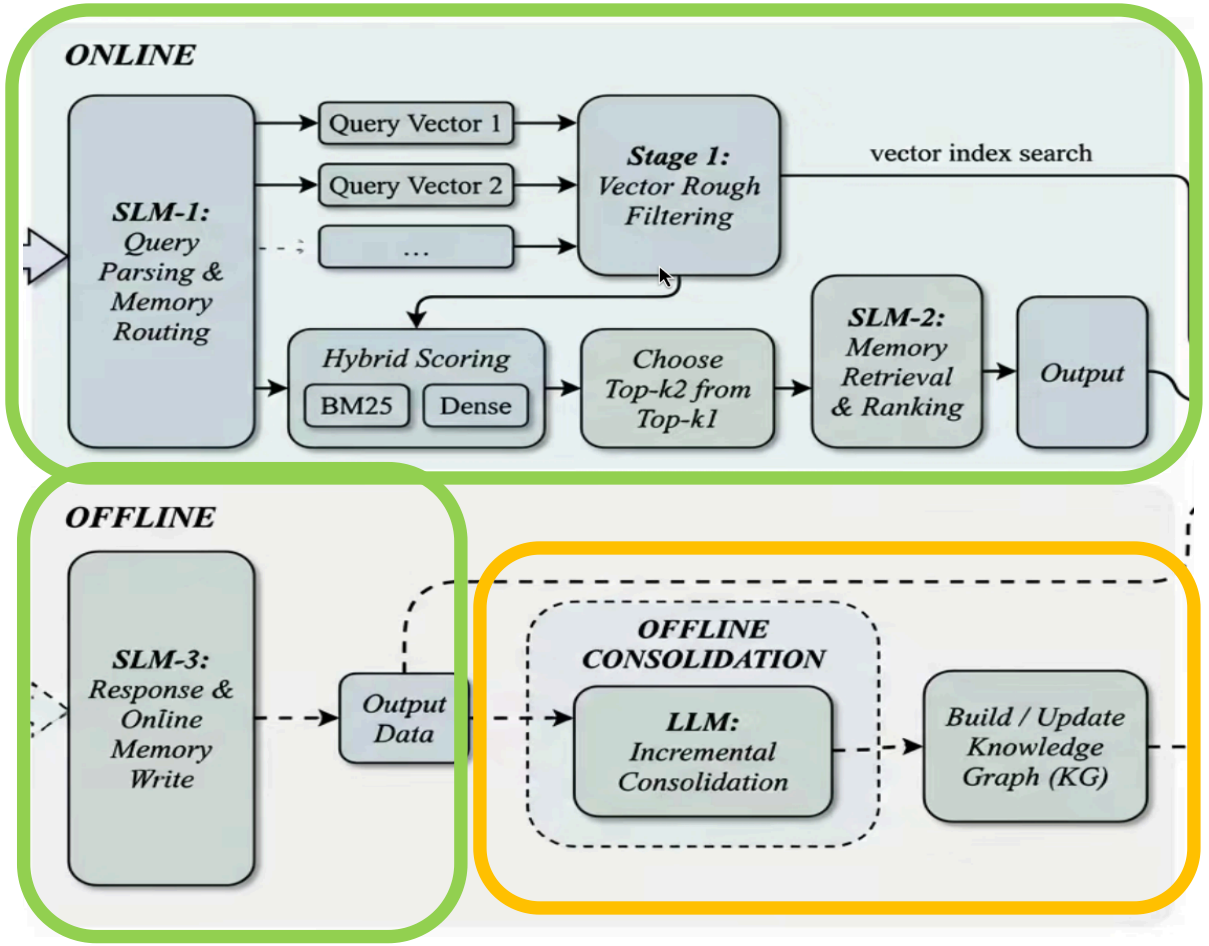
Figure 2: Multiple SLMs coordinate an online pathway for query-time routing and retrieval over STM/MTM, and an offline pathway that incrementally consolidates MTM into a graph-structured LTM.

The central idea: put each task on the right path

frequent, structured decisions to small language models (SLMs), and defers heavier knowledge consolidation to an offline LLM.

Online: plan queries → retrieve and filter → answer → write episodic memory.

Offline: abstract selected episodes → remove identity details → incrementally update shared knowledge.

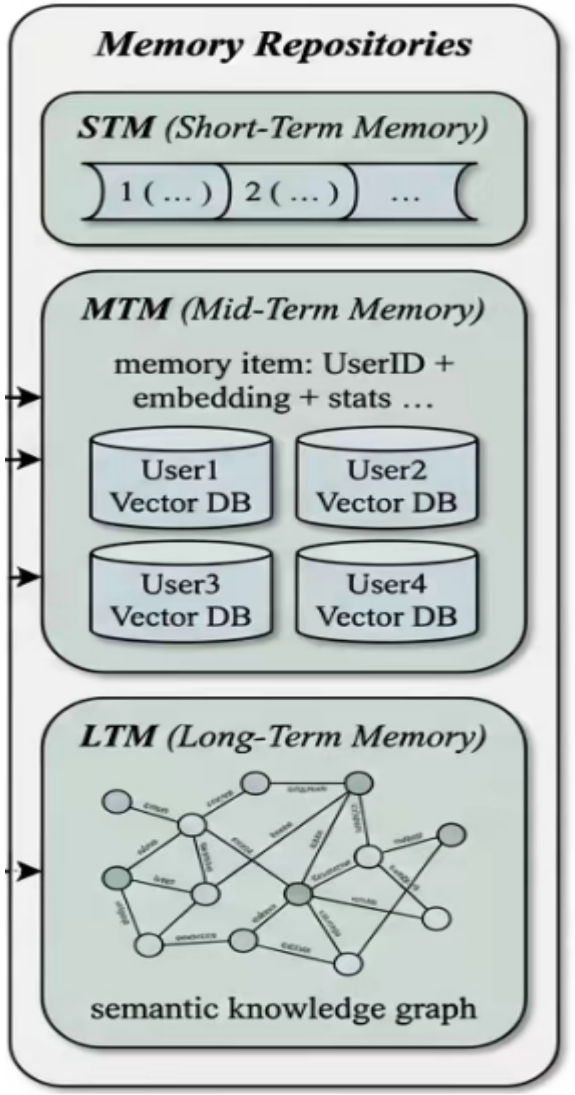


A three-level memory with two update timescales

STM · Short-term working context — recent turns in the active session; not persisted or externally retrieved.

MTM · Mid-term episodic memory — compact, user-scoped summaries with time, access statistics, and embeddings.

LTM · Long-term semantic memory — de-identified, shared knowledge organized as a lightweight graph.



MTM preserves "what happened to this user"; LTM captures "what is generally reusable."

Algorithm 1 LightMem Online Retrieval

Require: User query x_t, truncated context C_t, user memory M_u^{MTM}, global memory M^{LTM}, retrieval budget K
Ensure: Retrieved memory set R_t

Intent Modeling and Query Control (SLM-1)

Infer high-level intent signals from (x_t, C_t), including (i) degree of personalization and (ii) reliance on recent vs. long-term information.
Generate a set of HQs $\{q_t^{(i)}\}_{i=1}^{n}$.
Produce metadata constraints ϕ_t (e.g., user identifier and optional temporal filters).
Allocate retrieval quotas across memory layers under a fixed Top-K constraint.

Stage 1: Metadata-Constrained Coarse Retrieval

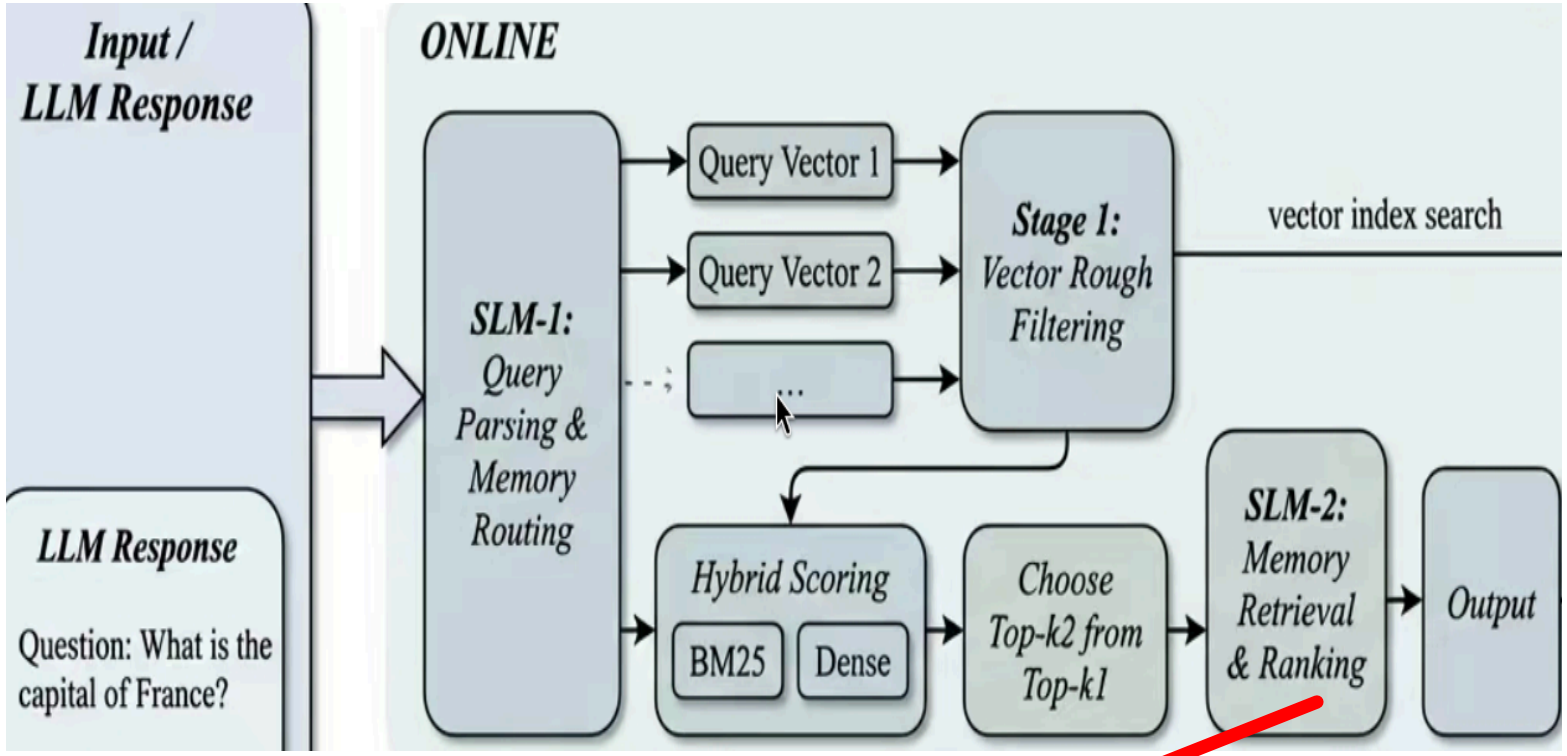
Initialize candidate set $C \leftarrow \emptyset$.
for each $q_t^{(i)}$, $i = 1, \ldots, n$ **do**
    Retrieve candidates from M_u^{MTM} and/or M^{LTM} using vector search under ϕ_t.
    Let $K_1^{(i)} \leftarrow \lceil \frac{2K}{n} \rceil$ and collect up to $K_1^{(i)}$ candidates as $C^{(i)}$.
    Update $C \leftarrow C \cup C^{(i)}$.
end for
Optionally truncate C to size $2K$ if $|C| > 2K$.

Stage 2: Semantic Filtering and Compression (SLM-2)

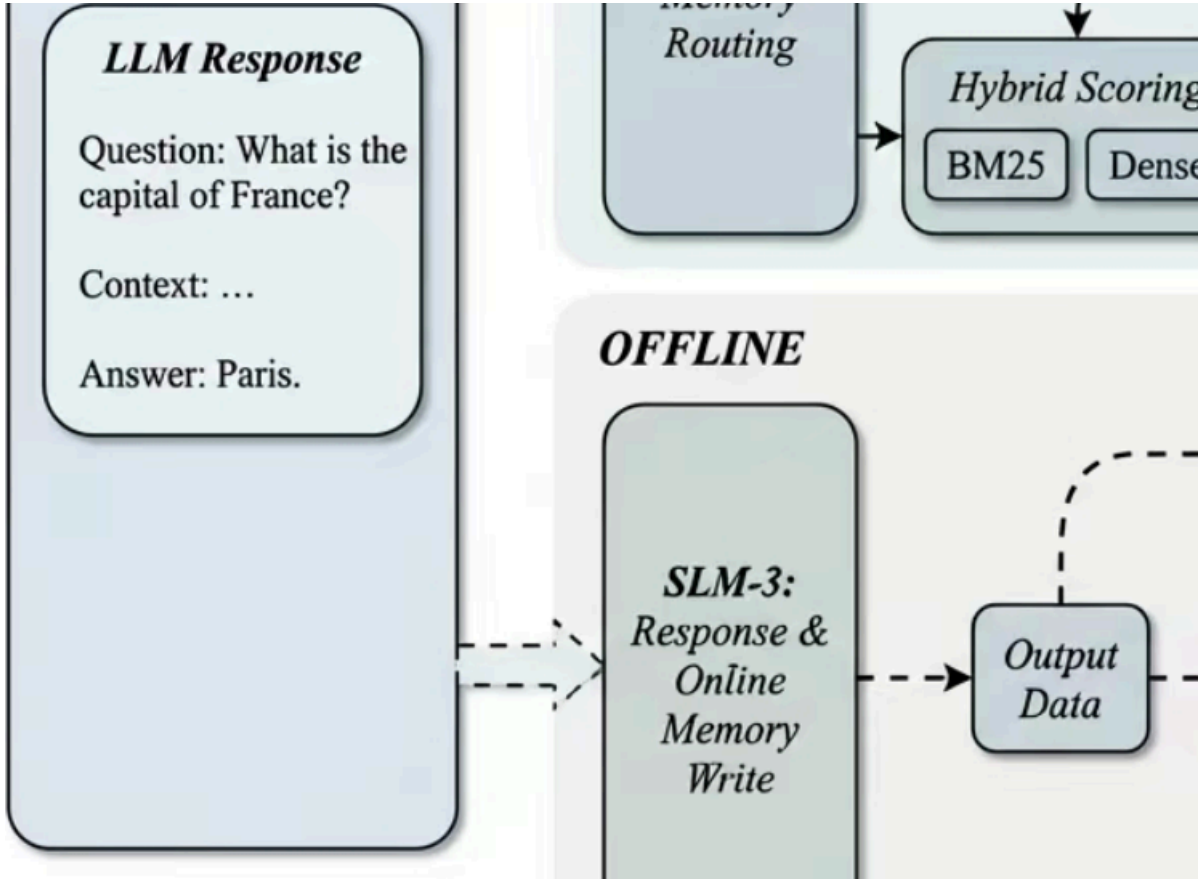
Perform semantic consistency checking and relevance assessment conditioned on $\{q_t^{(i)}\}$ and C.
Select up to K items to form the final retrieved set $R_t \subseteq C$ with $|R_t| \leq K$.

Output

return R_t (to be appended to the prompt for response generation).



SLM-2 is fine-tuned with LoRA on 2,000 constructed (Query,Subgraph, Path) samples



SLM-3 Writer: summarize the current interaction and maintain the user's MTM.

Issue commands only, no generation

Algorithm 2 LightMem Online Memory Writing and MTM Maintenance

Require: Current interaction (x_t, y_t, C_t), user mid-term memory M_u^{MTM}, capacity bound B
Ensure: Updated M_u^{MTM}

Memory Extraction and Compression (SLM-3)
Extract reusable, user-relevant information from the current interaction (x_t, y_t, C_t).
Compress the extracted content into a concise semantic memory summary.
Memory Appending
Append the compressed memory summary to the user-scoped MTM M_u^{MTM}.
Redundancy and Conflict Handling
Identify highly repetitive or semantically overlapping MTM entries and merge or rewrite them to reduce redundancy.
Resolve conflicting information using temporal cues and accumulated evidence strength.
Capacity-Bound Maintenance
if $|M_u^{\mathrm{MTM}}| > B$ **then**
    Evict stale or low-utility entries based on recency and access statistics.
    Further compress redundant content to enforce the capacity bound.
end if
return Updated M_u^{MTM}.

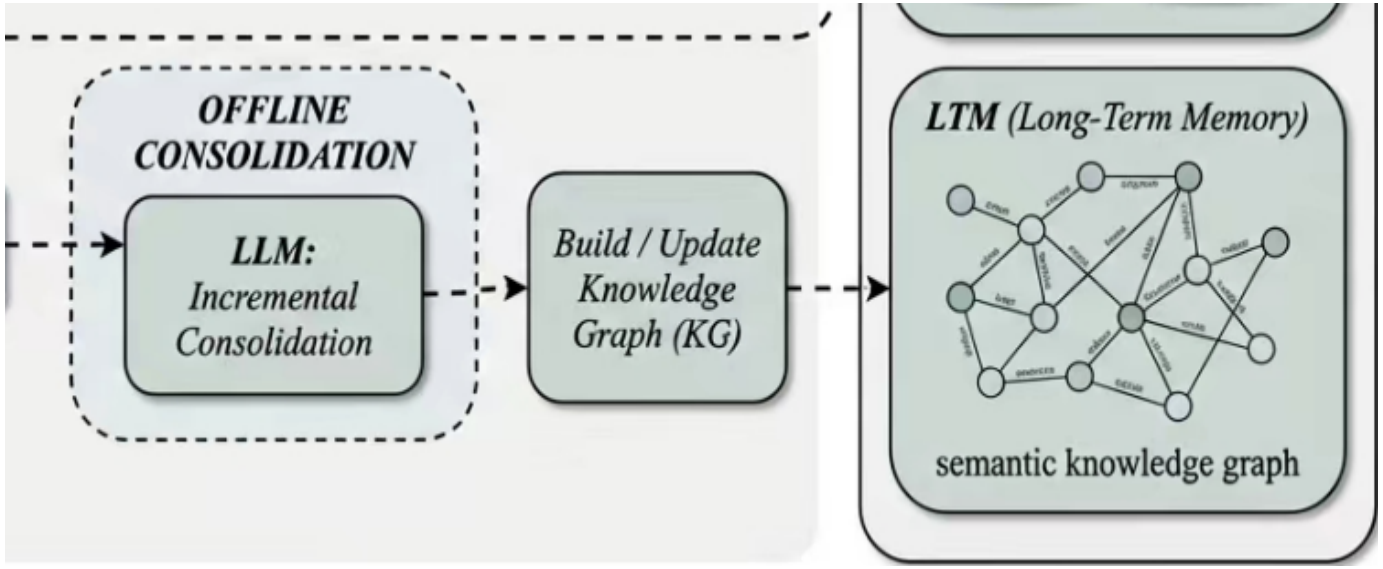


Table 9: Node and edge types used in the LTM knowledge graph.

Category	Type	Description and Example
Nodes	Entity	Concrete objects, locations, or named items extracted from interactions (e.g., "Python script", "Paris", "Project X").
	Concept	Abstract categories or generalized classes that characterize shared properties of entities (e.g., "Programming Language", "Capital City", "Urgent Task").
Edges	IsA	Hierarchical membership relation between an entity and a concept (e.g., *Paris* $\xrightarrow{\text{IsA}}$ *Capital City*).
	HasProperty	Links an entity to a specific attribute or state (e.g., *Project X* $\xrightarrow{\text{HasProperty}}$ *Completed*).
	RelatedTo	Generic semantic association based on co-occurrence or functional relatedness (e.g., *SNN* $\xrightarrow{\text{RelatedTo}}$ *Neural Operator*).
	Implies	Logical or causal dependency extracted from reasoning traces (e.g., *High Density* $\xrightarrow{\text{Implies}}$ *Congestion*).

Algorithm 3 LightMem Offline Consolidation into LTM

Require: MTM items flagged for consolidation, existing LTM graph M^{LTM}
Ensure: Updated LTM graph M^{LTM}

Selection of Candidate Episodes

Select MTM items that are newly written, frequently retrieved, or marked as low-utility under MTM capacity pressure.

Abstraction and De-identification

Abstract episodic interaction records into user-agnostic semantic knowledge units.
Remove personal identifiers and session-specific details.

Graph Insertion and Update

Perform similarity search over M^{LTM} to identify nearby semantic anchors.
Insert new nodes and edges within the local neighborhood of matched anchors.
Merge or update existing nodes when semantic overlap exceeds a threshold.

Evidence Accumulation and Forgetting

Aggregate supporting evidence across consolidation cycles.
Apply confidence decay to weakly supported knowledge units.
Remove or down-weight stale or incidental knowledge.
return Updated M^{LTM}.

Evaluation: long-horizon dialogue, varied backbones

LoCoMo: long conversations (about 9K tokens on average); single-hop, multi-hop, temporal, open-domain, and adversarial questions.

DialSim: 1,300 multi-party sessions spanning years, based on TV-show dialogue.

Baselines: full-context LoCoMo, ReadAgent, MemoryBank, MemGPT, and A-MEM.

Backbones: GPT-4o / GPT-4o-mini, Qwen2.5-1.5B / 3B, and Llama-3.2-1B / 3B.

Primary measures: F1, BLEU-1, context tokens; DialSim also reports ROUGE, METEOR, and SBERT semantic similarity.

LoCoMo: gains across model scales and reasoning types

Backbone	Method	Multi-hop F1	Temporal F1	Adversarial F1	Effective Context Length
GPT-4o-mini	A-MEM	27.02	45.85	50.03	2520
GPT-4o-mini	LightMem	**28.85**	**46.20**	**54.50**	**1150**
GPT-4o	A-MEM	32.86	39.41	36.35	1216
GPT-4o	LightMem	**34.50**	**40.10**	**41.50**	**680**
Qwen2.5-1.5B	A-MEM	18.23	24.32	46.00	1300
Qwen2.5-1.5B	LightMem	**21.50**	**25.60**	**49.80**	**720**
Llama-3.2-1B	A-MEM	19.06	17.80	58.81	1376
Llama-3.2-1B	LightMem	**22.40**	**19.90**	**63.50**	**750**

DialSim: stronger lexical and semantic alignment

Table 3: Comparison of different memory mechanisms on DialSim with GPT-4o-mini.

Method	F1	BLEU-1	ROUGE-L	ROUGE-2	METEOR	SBERT
LoCoMo	2.55	3.13	2.75	0.90	1.64	15.76
MemGPT	1.18	1.07	0.96	0.42	0.95	8.54
A-MEM	3.45	3.37	3.54	3.60	2.05	19.51
LightMem	**4.12**	**3.95**	**4.20**	**4.15**	**2.48**	**23.40**

Latency: a quality–efficiency balance

Table 4: Scalability and latency analysis across five baselines on GPT-4o-mini.

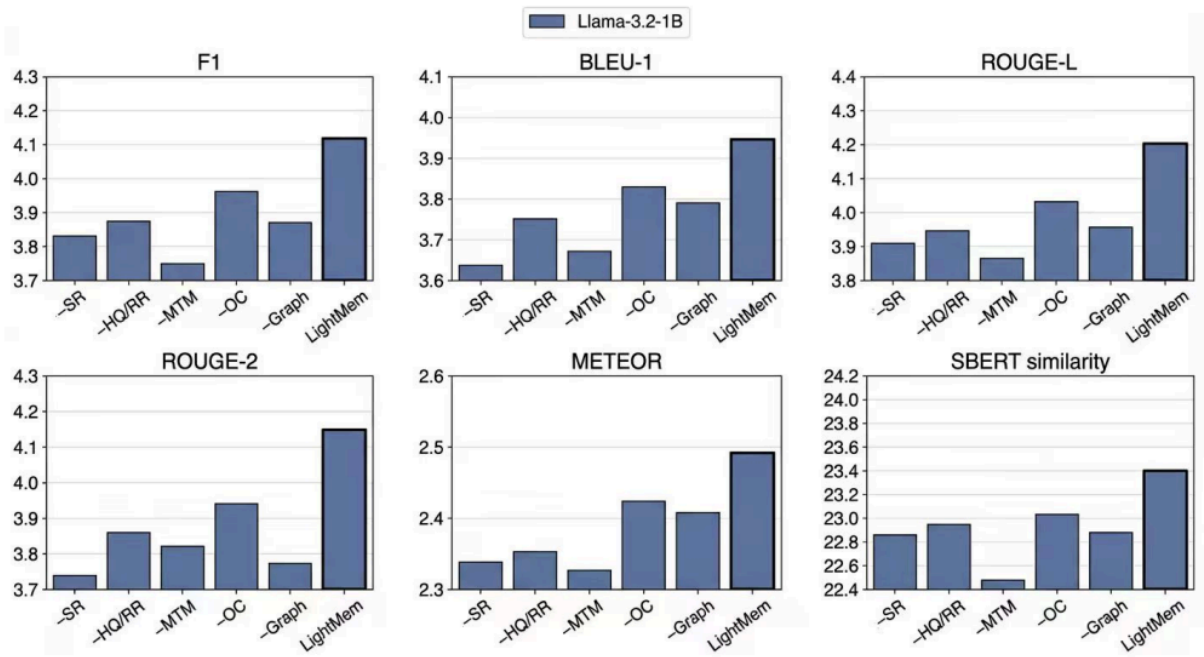
Method	Retrieval Latency P50 (ms)↓	Retrieval Latency P95 (ms)↓	End-to-End P50 (ms)↓	End-to-End P95 (ms)↓
LoCoMo	0	0	2054	3658
ReadAgent	34	97	614	1027
MemoryBank	16	51	439	1047
MemGPT	143	451	2087	3451
A-MEM	856	1583	914	3682
LightMem	83	167	581	1325

The claim is balance: LightMem is not the fastest retriever, but combines stronger quality with bounded, competitive latency.

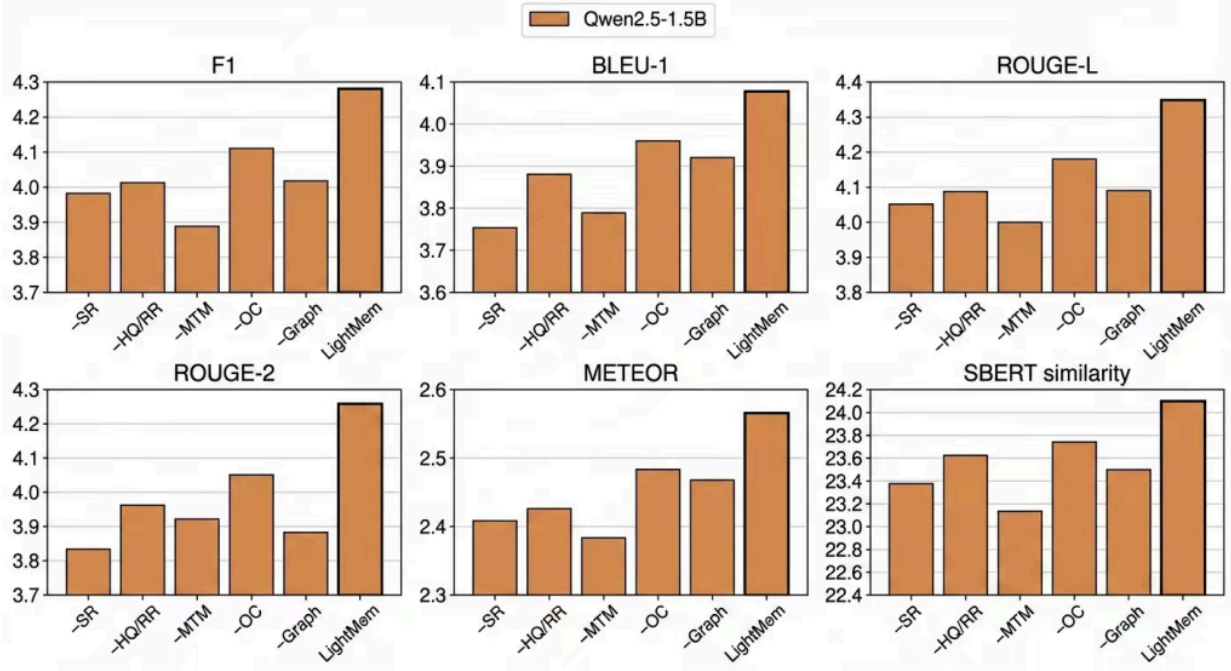
Table 5: Performance stability under natural MTM growth on DialSim with Llama-3.2-1B. Statistics are computed cumulatively along the same full dialogue trajectory at different MTM sizes.

MTM size	Vector retrieval	LightMem	ΔF1
∼100	3.95	3.98	0.03
∼1,000	3.90	4.05	0.15
∼5,000	3.86	4.09	0.23
10,000	**3.83**	**4.12**	**0.29**

Ablations: the gains come from complementary parts



(a) Ablation study on Dialsim with Llama-3.2-1B



(b) Ablation study on Dialsim with Qwen2.5-1.5B

Figure 3: Ablation study on DialSim. We report F1, BLEU-1, ROUGE-L, ROUGE-2, METEOR, and SBERT similarity.

Each component contributes; episodic MTM and online query/filtering show especially direct gains.

Stress tests reveal where memory systems can fail

Table 6: Error injection stress test on DialSim under a fixed final Top-K memory budget.

Group	Setting	F1	SBERT
A	Full LightMem	**4.12**	**23.40**
B	SLM-1 50% HQ noise	3.95	23.08
C	w/o SLM-2 reranking	3.83	22.56
D	SLM-3 50% write noise	3.78	22.45
E	Cascading failure	1.85	11.20

Table 7: Update gap stress test on the LoCoMo multi-hop subset with GPT-4o-mini.

Group	Setting	Multi-hop F1
A	Full LightMem (normal gap)	**28.85**
B	LTM-only retrieval	19.45
C	MTM-only retrieval	20.12
D	MTM noise saturation	23.10

The paper notes a design limitation: alternative consolidation and online-control policies remain underexplored.

Takeaway: memory quality depends on the full lifecycle—bad writes can become persistent pollution.

What is genuinely new here?

1 · SLM-specialized online memory operations — separates query planning, candidate selection, and writing into bounded, structured tasks.

2 · Two-stage retrieval — combines high-recall vector search with semantic evidence verification under a fixed 2K→K budget.

3 · Online/offline decoupling — keeps latency-sensitive work online while consolidating reusable knowledge asynchronously.

4 · Episodic/semantic separation — user-scoped MTM and de-identified graph LTM are retrieved together to bridge recent and older knowledge.

The contribution is a coordinated system design, rather than simply replacing a large model with a small one.

Discussion: strengths, caveats, and open questions

LightMem makes long-term agent memory more practical by matching model size and execution time to the operation.

Strength: modular pipeline controls candidate noise and keeps online retrieval bounded.

Strength: gains are reported across two benchmarks and multiple backbone scales, alongside context and latency measurements.

Caveat: strongest performance does not mean best score in every category;

Caveat: the system adds modules, prompts, SLM-2 LoRA training, metadata, and offline graph maintenance.

Discussion:

- How should systems calibrate HQ budget allocation?
- How can write errors be detected before they persist?

Core insight: good agent memory is a lifecycle and retrieval-design problem—not just a larger context window.